FLUXION-TIDE · RESEARCH NOTES

The landscape

Where we stand, where the points are, what the generator is missing, and how we find out why the model fails

Prepared for Naresh · 25 October 2026 · ft3_main (29M), fev-bench, TIME, synthetic generator clean2

43.8
our fev-bench skill (groups); the leader has 51.8

69%
of a 22M model's gap comes from tasks with covariates

7.4%
of our training lanes are targets with known covariates

132k
series in the planned benchmark-free dev slice

What this document says, in five sentences

1. We are 8 skill points behind the fev-bench leader and 23rd of 40 on TIME; model size is not the lever (our 100M was no better than the 30M, and a 5M model beats us).
2. For models at our level, most of the missing points sit in a few dozen tasks, and most of those come with covariates (weather, prices, holidays, opening days).
3. Our generator is strong on seasonality, repetition and textbook dynamics, but weak where real use lives: covariates that drive a target, real calendars, history that shows a cycle only once or twice, breaks, and true gaps.
4. We now have a benchmark-free catalog of 377 real datasets to check every change against, and a method to find *why* a forecast failed, not just that it did.
5. The plan: cheap switches first, then covariate scenarios with a causal-graph generator and a calendar engine, tested at 5M before a single confirmation at 30M.

How to read this. Blue boxes explain an idea in plain words. Amber boxes are rules to keep. Green boxes are directions worth exploring. Every number comes from our own runs, the published fev-bench result files, or the project notes; estimates are marked as estimates.

Contents

1 Where we stand **2**

2 The three scoreboards **3**

3 Where the missing points are **4**

4 Our generator today **6**

5 What the generator is missing **8**

6 Three designs for the main bets **11**

6.1 Covariate scenarios 11

6.2 A causal-graph covariate generator 11

6.3 A calendar engine 12

7 Real data for checking our work **12**

8 Finding out why a forecast fails **13**

8.1 Step 1: fixable error 14

8.2 Step 2: attribution by repair 14

8.3 Step 3: which series properties predict failure 14

8.4 Step 4: confirm by intervention 14

9 Changes on the model side **15**

10 The plan **15**

10.1 First experiment: separate data from model 15

10.2 Order of work 16

11 Rules to keep **16**

12 Ideas worth exploring **17**

A Glossary and sources **18**

1 Where we stand

Our best model, `ft3_main` (29M parameters, trained only on synthetic data), scores **43.78** on fev-bench with its covariate (group) mode on and 42.09 with it off. The leader, TimeRSI-1.5, scores **51.8**. On TIME we are 23rd of 40 at a normalised MASE of 0.680; Chronos-2 is at 0.662 and Timestral-5M at 0.652.

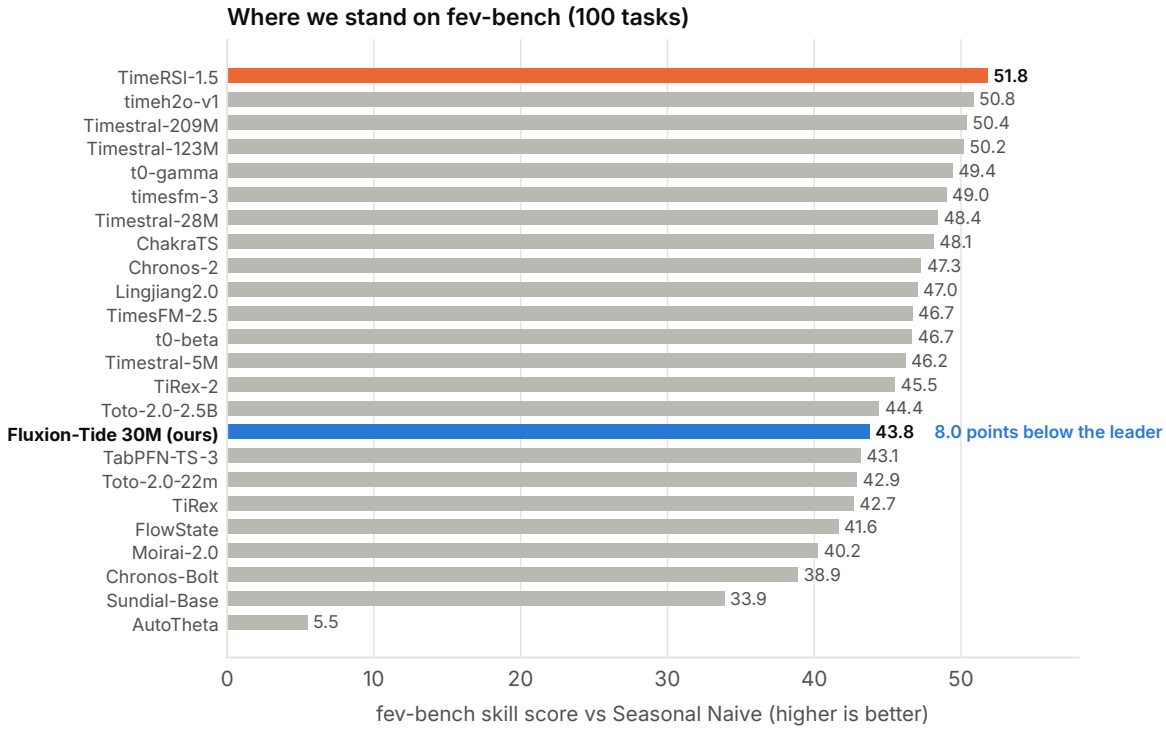


Figure 1: fev-bench skill scores computed from the result files published in the fev repository (higher is better). Our row is the owner's own scoring of ft3_main with groups on; the other rows use the published per-task files. Naive and five models with missing tasks are left out.

> **In plain words** A **skill score** says how much smaller a model's error is than a very simple forecaster (Seasonal Naive: "repeat the last cycle"). A skill of 44 means our error is about 56% of Seasonal Naive's, averaged in a way that treats every task equally. Going from 43.8 to 51.8 means cutting our error by about 15% across the board, which is a lot.

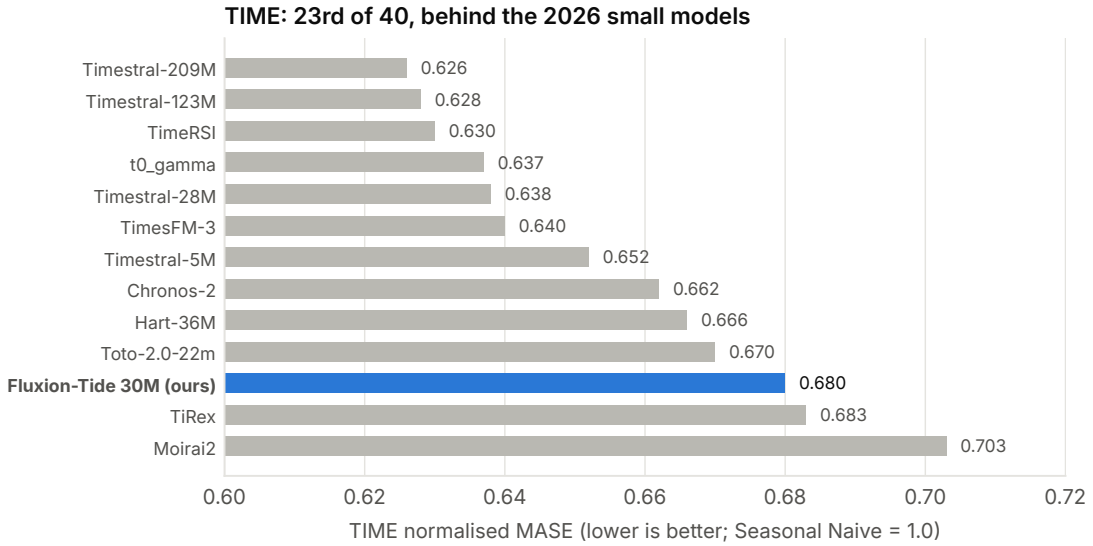


Figure 2: TIME (50 fresh datasets, 98 tasks) normalised MASE, excerpt of the 40-model board from our notes (TIME_FULL_LEADERBOARD). Lower is better.

Keep in mind

- Against the top models the gap is **broad**: we beat Chronos-2 on 24% of TIME tasks and Timestral-28M on 9%.
- Size is not the lever: our 100M was $+0.21$ skill over the 30M on fev (not significant), and Timestral-5M beats our 30M on about 90% of TIME tasks.
- Real-data post-training gave about $+0.5$ skill, but that pool was 96% weather. It shows *that* data did not help much, not that real data cannot.

2 The three scoreboards

Three benchmarks decide how a time-series foundation model is judged. They overlap in style but not in data, and each has rules for what may touch training.

	TIME	fev-bench	GIFT-Eval (test)
Size	39 datasets, 50 dataset/frequency entries, 98 tasks	100 tasks over 59 dataset configurations	28 datasets, 97 configurations
Character	fresh data (2025), mostly FRED economics, energy, IMOS ocean, Singapore city data, Azure 2019	energy, retail and cloud heavy; 42-46 tasks give covariates	classic sets: M4, electricity, solar, traffic, ETT, hospital
Metric	MASE and CRPS / Seasonal Naive, geometric mean	SQL skill vs Seasonal Naive, win rate	MASE, CRPS ranks
How we use it	final check only	final check; per-task files for analysis	not used for selection

Table 1: The benchmarks. Lists were read from the benchmark repositories on 25 October 2026 (TIME `datasets.yaml`, fev `tasks.yaml`, GIFT-Eval notebooks).

Why this matters If a dataset from a benchmark, or the same series published somewhere else, ends up in what we tune on, our score on that benchmark stops meaning anything. Several TIME datasets are FRED republications, so "FRED" as a whole is unsafe unless we remove those exact series. The same goes for crypto prices, Singapore weather and PM2.5, EIA crude and product prices, and the 2019 Azure VM trace.

Rules for the scoreboards

- Never choose a model, a generator setting or a mix weight by looking at TIME. Keep a count of how many times we look at fev.
- Tune on the dev catalog (Section 7), confirm on a locked part of it, and score fev and TIME at the end.
- fev differences below about 0.3 skill points are mostly noise; always use paired confidence intervals.

3 Where the missing points are

The published fev result files let us split the gap between a model at our level and the leader, task by task. Two facts stand out.

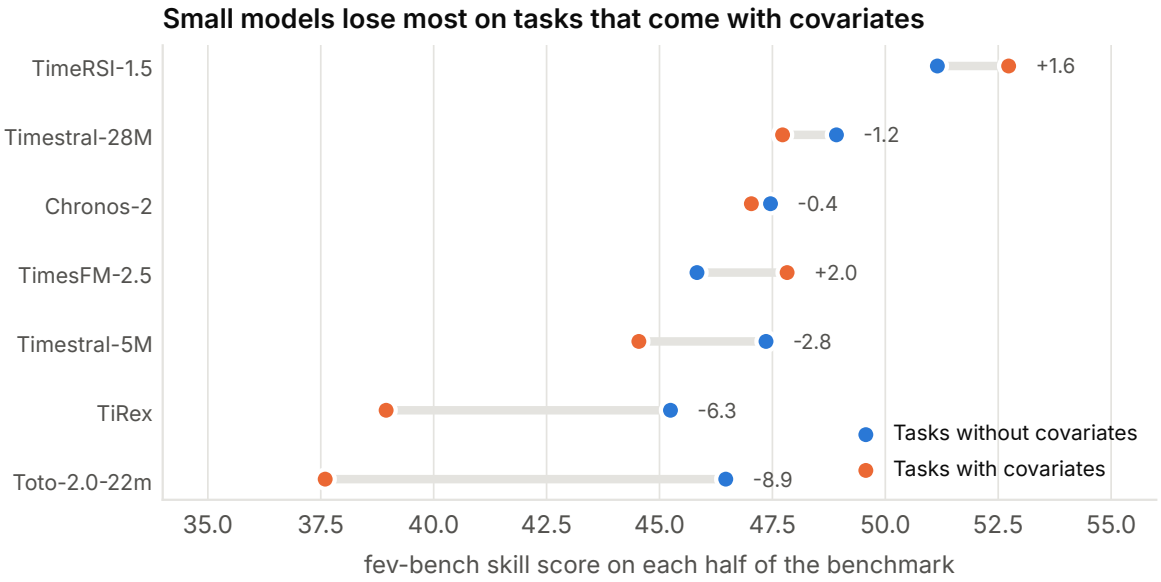


Figure 3: Skill on the tasks without covariates (blue) and with covariates (orange). The number is the difference. Small models fall off sharply when covariates are present; the leader does not.

Fact one: covariates. For Toto-2.0-22m, a model at our level, 69% of the gap to TimeRSI-1.5 comes from the tasks that supply covariates. If it matched the leader on those tasks alone it would score 49.2 instead of 42.9. For Timestral-5M the share is 61%, for Chronos-2 53%.

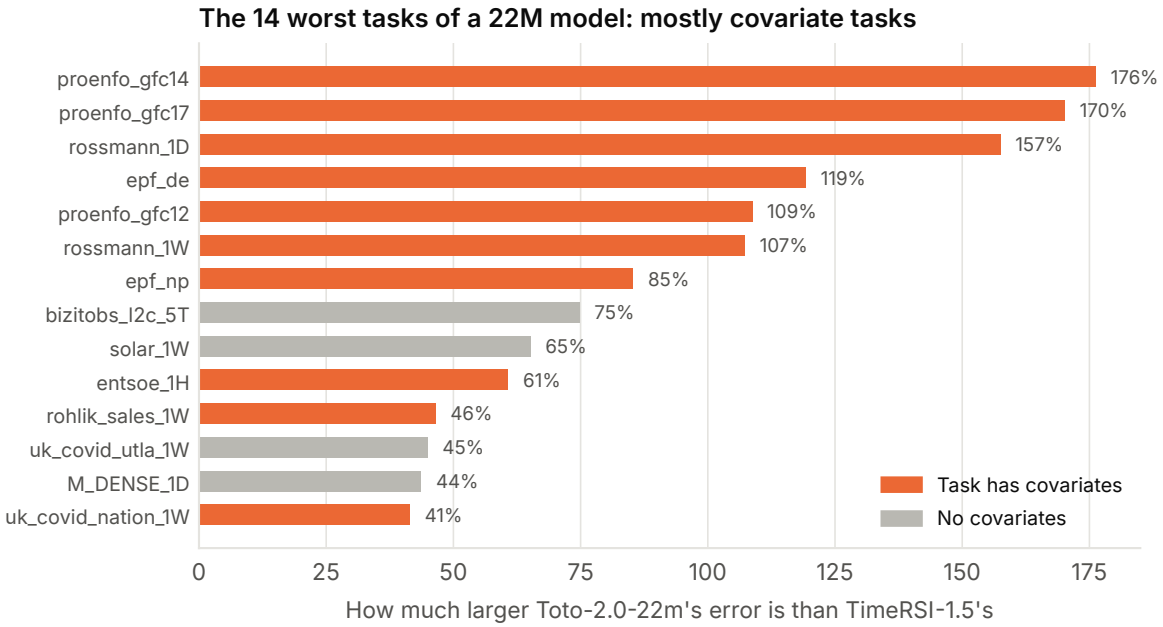


Figure 4: The 14 tasks where Toto-2.0-22m is furthest behind the leader. Ten of them come with covariates: GEFCom load with temperature, Rossmann sales with opening days, promotions and holidays, electricity prices with load and wind forecasts, ENTSO-E load with radiation and temperature.

Fact two: concentration. Ten tasks hold 42% of the gap and 25 tasks hold 70%. We do not need to be a little better everywhere; we need to stop losing badly in a few recognisable places.

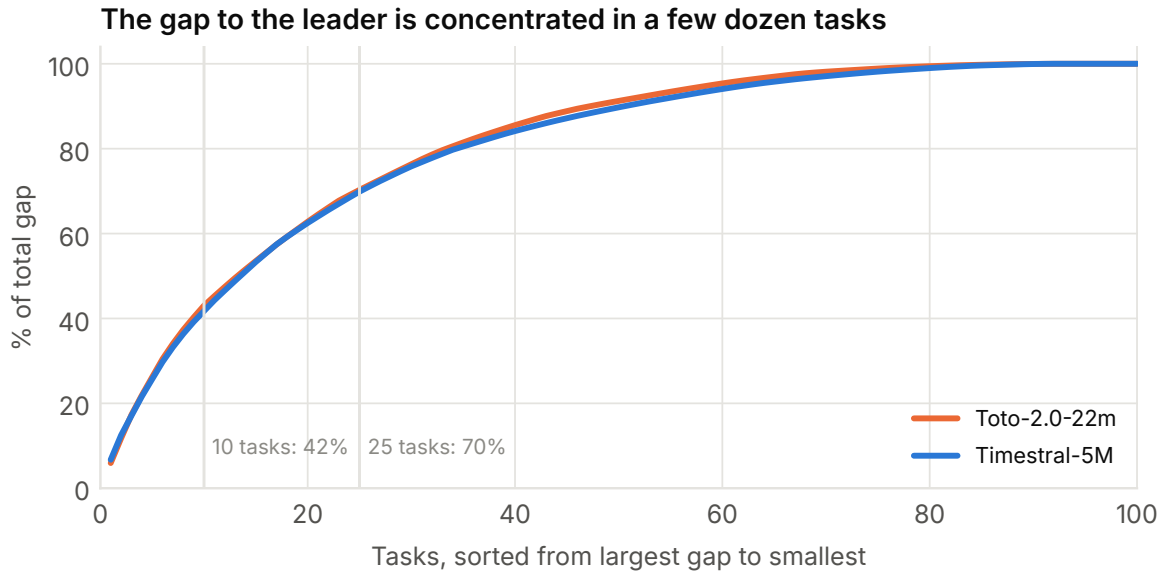


Figure 5: Cumulative share of the gap to TimeRSI-1.5, tasks sorted from the largest gap down. The two models have almost the same curve.

What a covariate is A covariate is another series that helps explain the one we forecast. Some are **known in the future** (the temperature forecast, a planned promotion, whether the shop is open, a public holiday); some are only **known in the past** (yesterday's number of customers). A model that can read them correctly gets a large advantage on exactly the tasks where we lose most.

Not yet measured for us These splits use other models' published files. We have not yet run the same split on ft3_main's own per-task file. Our group mode adds 1.7 skill points (43.78 vs 42.09); Chronos-2 reports about 16% lower error on known-covariate tasks, our earlier v4 about 4%. Running the split on our own file is the first thing to do.

4 Our generator today

ft3_main learned only from synthetic series. The generator (`tidemark/real/ft3_core.py`, profile `clean2`) draws a base family, then stacks layers on top. I ran it on CPU (2,048 series at each of four lengths) to measure what it actually produces.

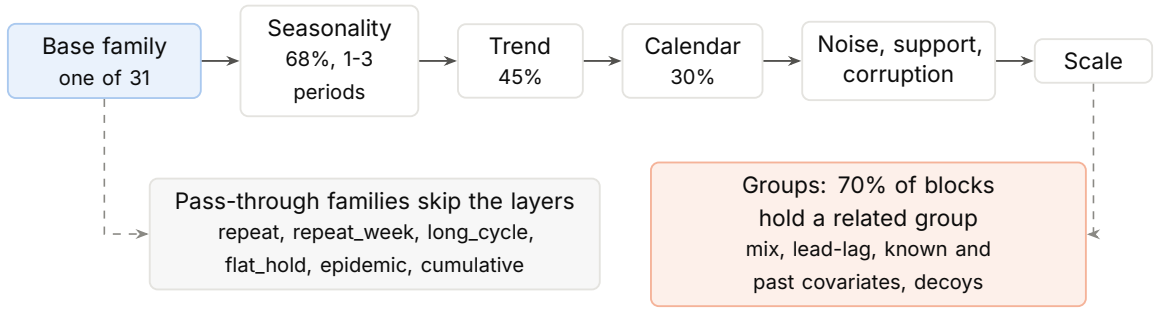


Figure 6: How one synthetic series is made. Percentages are the share of eligible rows that receive a layer.

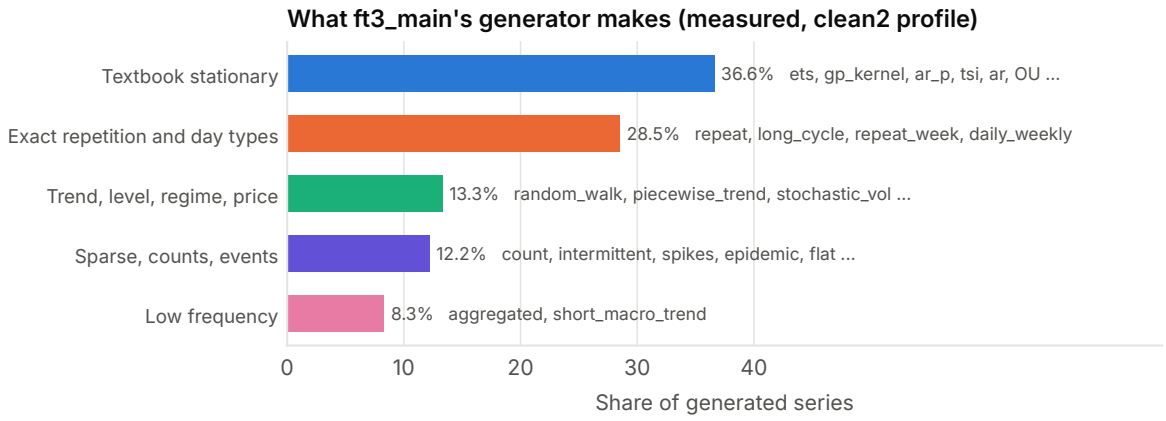


Figure 7: Share of generated series by role, measured from the family weights of the clean2 profile with the ft3 additions (agg, periods, epidemic, repeat, day types, weekday, yearly).

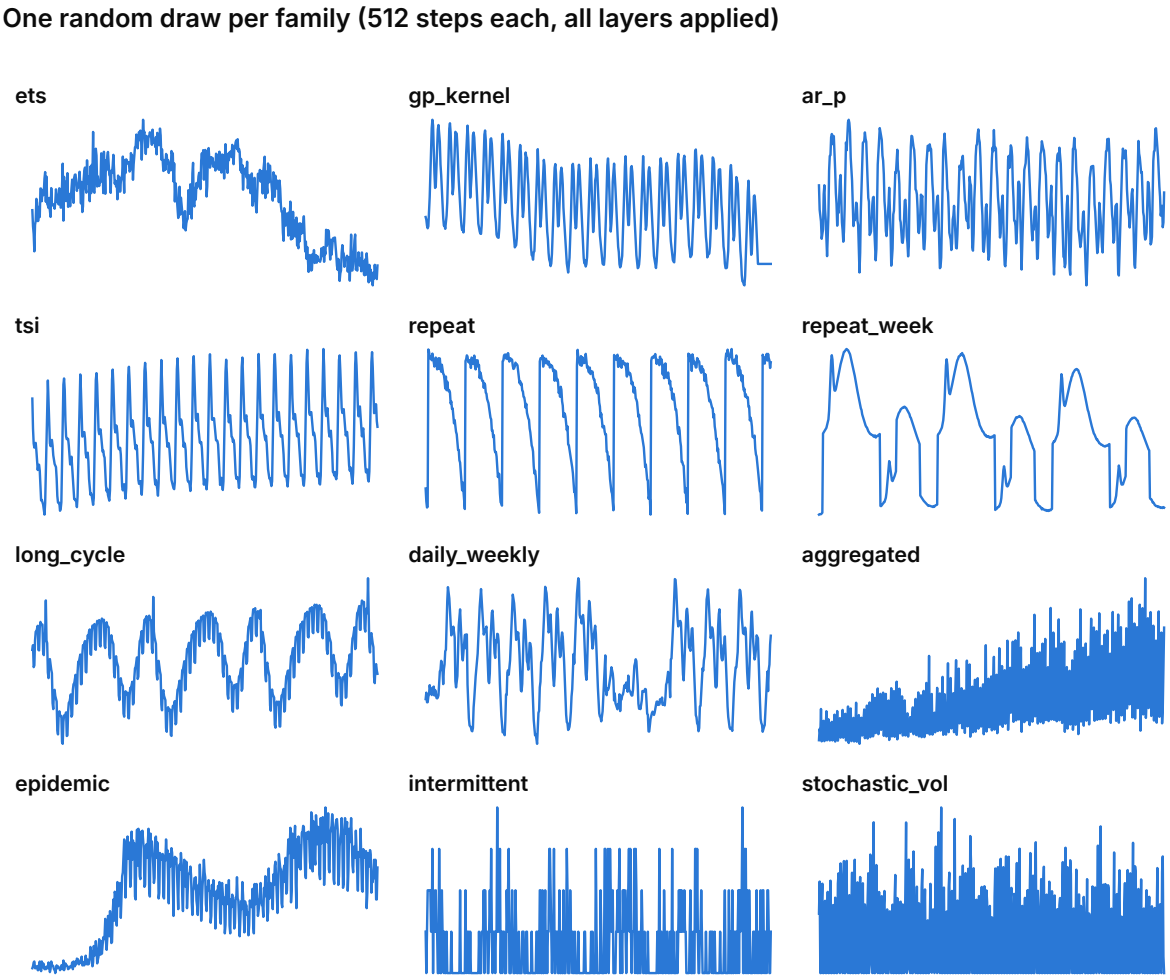


Figure 8: One random draw from each of twelve families, after all layers. The generator already covers smooth stationary dynamics, exact repetition, day types, long cycles, aggregation, epidemics, intermittency and volatility.

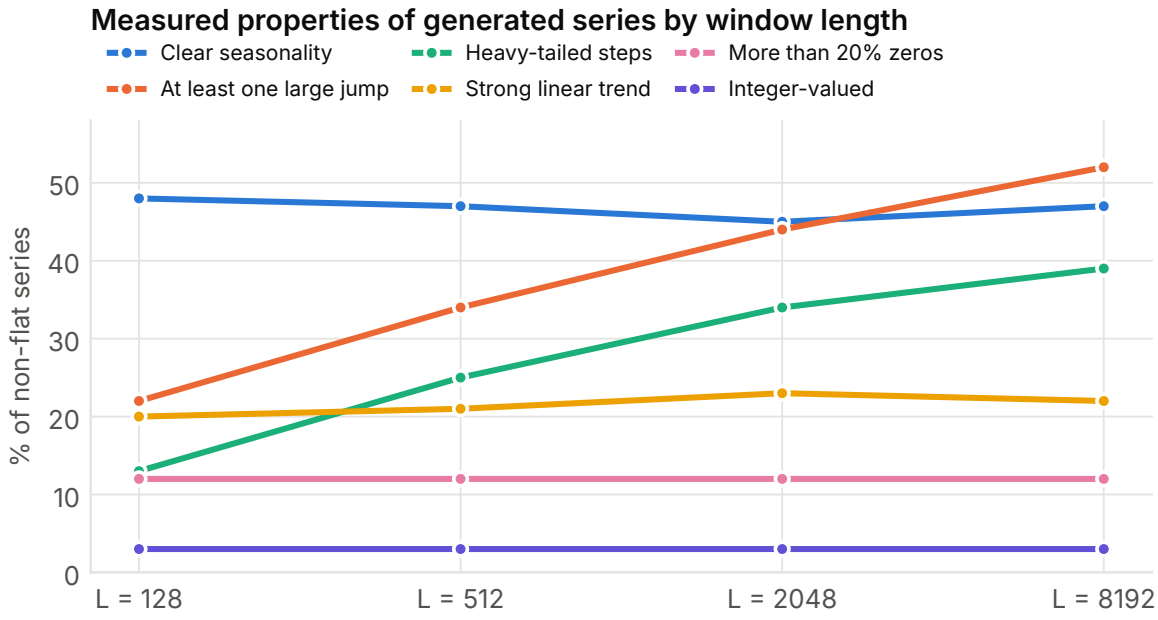


Figure 9: Measured properties by window length (% of non-flat series). About half the series are clearly seasonal at every length; only 3% are integer-valued.

What the generator does well It was built against measured fev-bench gaps, round after round: non-sinusoidal seasonal shapes, nested periods, exact repetition at any period, weekday versus weekend profiles, low-frequency series made by aggregating daily ones, epidemic waves, flat series that stay flat, cumulative counters, and covariate groups with decoys and on/off gates. That is why a synthetic-only 29M model is level with TiRex and ahead of every 2024-25 model.

Assumptions in this measurement The RUN section that trained ft3_main is not in the repository, so I used the defaults in `CONFIG` plus `yearly` and `cov_r3` on (the repo notes say ft3_main used year generators and round-3 covariates). Print `torch.load("ft3_main.pt")["cfg"]` to confirm. The seasonality detector only checks lags up to a third of the window, so it misses long_cycle's 2-4-cycle periods.

5 What the generator is missing

Each gap below has four parts: what is missing, the evidence, the change, and how to check it. "Dev" names real datasets from our benchmark-free catalog.

#	Gap	Kind of work	Priority
1	**Covariate realism and volume, with a causal-graph generator**	new generators	main bet
2	**A real calendar engine**	new generator layer	main bet
3	**History that shows a cycle only once or twice**	reweight + calendar	high
4	**Structural breaks and regime change**	switch exists, off	quick win
5	**Drifting and quasi-periodic cycles**	switch exists + one family	quick win
6	**True missing values (and an inference bug)**	small change	quick win
7	**Counts, intermittency, product life cycles**	one family	high
8	**Hierarchies**	group kind	later
9	**Nonlinear dynamics (ODE systems)**	offline pool	later
10	**Financial tails**	one family	later

Table 2: Summary of the gaps, ranked by expected effect on fev and TIME (judgement informed by the measurements above).

1 Covariate realism and volume

MAIN BET

MISSING A continuous covariate is just another synthetic series, and its effect is a fixed curve (linear, ReLU, tanh, step or V) that never depends on time of day or season. The real patterns that cost the most are not generated as such: temperature driving load differently at each hour; price set by residual load (load minus wind and solar) through a steep supply curve; sales shaped by open days, promotion windows, price and typed holidays; PV output following irradiance with inverter clipping. There are no static or categorical covariates at all, and no generic causal-graph generator of the kind t0 uses.

EVIDENCE Covariate tasks hold 53-69% of small models' gap on fev. Our group mode adds only 1.7 points. The holiday-flag probe stayed near 0.05 for every twin arm. Only 7.4% of training lanes are covariate-driven targets.

CHANGE Scenario generators that draw a target and its covariates from one mechanism (Section 6), plus a causal-graph generator for breadth. Hand known covariates over with forecast error, raise covariate-driven targets to 20-30% of lanes, with 1-13 covariates per group.

CHECK Dev: EIA-930 load with GHCN or ERA5 temperature; SMARD and ENTSO-E prices with load and renewables (not the EPF countries); PV with NASA POWER irradiance; Paris bike counts with

weather; Iowa liquor sales with holidays. Target: at least 10% lower error with covariates than without, where the mechanism exists.

2 A real calendar engine

MAIN BET

MISSING Series have no dates. The calendar layer is a weekend effect plus random holiday days (2%). Months are 30 steps except in the aggregation family. No moving festivals (Easter, Diwali, Eid moving about 11 days a year, Lunar New Year), no daylight-saving 23- and 25-hour days, paydays, month or quarter ends, school terms, bridge days or the Black Friday to Christmas run-up. The model has no calendar input.

EVIDENCE Holiday probes near 0.05; Rossmann and Rohlik among the worst fev tasks.

CHANGE Give each series a start date and a frequency; derive weekdays, months, holidays and school terms from real tables for 15-20 countries (the `holidays` package would be a new dependency). Apply effects with lead and lag windows. Show the calendar as known covariates in about half the rows and hide it in the rest.

CHECK Dev: TSA throughput, Eurocontrol traffic, Iowa liquor, Agmarknet mandi prices (Diwali, monsoon), Seattle library checkouts.

3 History that shows a cycle only once or twice

HIGH

MISSING Only long_cycle (9.5% of series) shows a long period one to six times. Every other seasonal family needs two or three full cycles in the window; about 94% of detected seasonal series show eight or more. No “same weekday last year” alignment, no partial-cycle history.

EVIDENCE fev history band 2049+: 0.520 vs Chronos-2 0.479, unchanged in every fine-tune. Year-once probe about 0.1. Patch probe: aligned single-occurrence recall 0.98 (Chronos-2 0.24), improving with size (100M: 0.905), so part of this is model capacity.

CHANGE long_cycle to about 15%, built on the calendar engine (day of year, leap years, weekday drift), mostly in contexts of 2,048 points or more.

CHECK Year-once and year-twice probes; dev daily series with 1-3 years of history (national park visits, CTA ridership, GHCN daily, library checkouts).

4 Structural breaks and regime change

QUICK WIN

MISSING The breaks layer exists (level, slope, variance, shock with recovery) but is switched off. Breaks only come through three families (about 6%). It was tested only inside an all-on run, never alone.

CHANGE Breaks on at 0.2, with some breaks in the last 10-30% of the context, plus a collapse-and-partial-recovery shape (2020).

CHECK Dev: Google mobility, TSA, Eurocontrol, CTA; GNSS station steps. Probe: a break at a controlled place; does the forecast start from the new level?

5 Drifting and quasi-periodic cycles

QUICK WIN

MISSING Seasonal drift (`tv_season`) is off. Nothing generates amplitude and frequency modulation, beats of two close periods, or cycles of irregular length.

CHANGE Switch it on and add one family: $x(t) = A(t)\, s(\varphi(t))$ with $\varphi(t) = 2\pi \int_0^t \mathrm{d}u / P(u)$ and a period that wanders a few percent; beats (tides: 12.42 h with 12 h); irregular cycles (sunspots 9-14 years, ENSO 2-7 years).

CHECK Dev: NOAA tide gauges, sunspots and F10.7, ENSO indices, BioDare2 circadian data, GNSS annual cycles.

6 True missing values

QUICK WIN

MISSING The corruption layer forward-fills its missing runs, so training never marks a single interior point as missing; only left padding and whole hidden patches are masked.

BUG The univariate forecast path runs `y = y[np.isfinite(y)]`: it deletes missing values and joins the pieces, which shifts every cycle after a gap. The group path keeps them masked.

CHANGE Train with scattered and block gaps marked as unobserved, including periodic ones (nights, weekends, market closures); at inference, keep the positions and mask them.

CHECK Count how many fev and TIME contexts contain missing values; dev: OpenAQ, NDBC buoys, USGS gauges.

7 Counts, intermittency, product life cycles

HIGH

MISSING About 3% of series are integer. No launch ramp, maturity, decline or death to zero; no stockouts (zeros that mean “no stock”, then a rebound).

CHANGE A demand family: low-count Poisson or negative-binomial draws on a seasonal, promotional intensity, Croston-style gaps between orders, a life-cycle envelope and stockout runs.

CHECK Dev: VN1 and VN2 competitions, Predict Future Sales, H&M, Seattle library, NHS prescribing.

8-10 Lower priority

LATER

HIERARCHIES Groups mix shared factors but nothing sums: add groups where store series add up to region and total (dev: Iowa liquor by store and county, Eurostat regions).

DYNAMICS No ODE systems (Lotka-Volterra, seasonally forced SIR, van der Pol, Duffing). Generate them offline into a pool, since time stepping is sequential.

FINANCE Jumps, the leverage effect and business-day gaps. The right forecast is close to naive with honest intervals, so check calibration, not point skill.

6 Three designs for the main bets

6.1 Covariate scenarios

Each scenario draws the target and its covariates from one mechanism, so the relation the model must learn is the one real data has. The equations are sketches; each parameter is drawn at random per series.

Scenario	Mechanism	Real example
Weather to load	$y_t = b_t + \beta_{h(t)} \max(0, T_t - T_c) + \gamma_{h(t)} \max(0, T_h - T_t) + \varepsilon_t$; temperature T has its own daily and yearly cycle; the hourly weights β_h, γ_h make heating and cooling act at different hours	GEFCom, ENTSO-E, EIA-930
Residual load to price	$p_t = a + b\, e^{c\,(L_t - W_t - S_t)} +$ spikes; load, wind and solar forecasts given as known covariates with forecast error	electricity price markets
Sales	$y_t = o_t \left(b_t \left(1 + \lambda\, \pi_t - \delta\, \pi_{t-k}\right) \left(p_t / \bar{p}\right)^{-\eta} H_t\right)$; open flag o_t, promotion π_t with a post-promotion dip, price elasticity η, typed holidays H_t	Rossmann, Rohlik, M5-like retail
Solar output	$y_t = \min\left(C,\ \eta\, I_t \left(1 - k\left(T_t - 25\right)\right)\right)$; irradiance I zero at night, clipping at capacity C	PV systems with irradiance forecasts
Events to traffic	baseline with day types plus event bumps; event calendar as a known flag	bike counters, transit ridership

6.2 A causal-graph covariate generator

Hand-built scenarios give realistic shapes but only for the cases we think of. A random causal graph gives breadth: many shapes of "this series drives that one", including series that look related but are not.

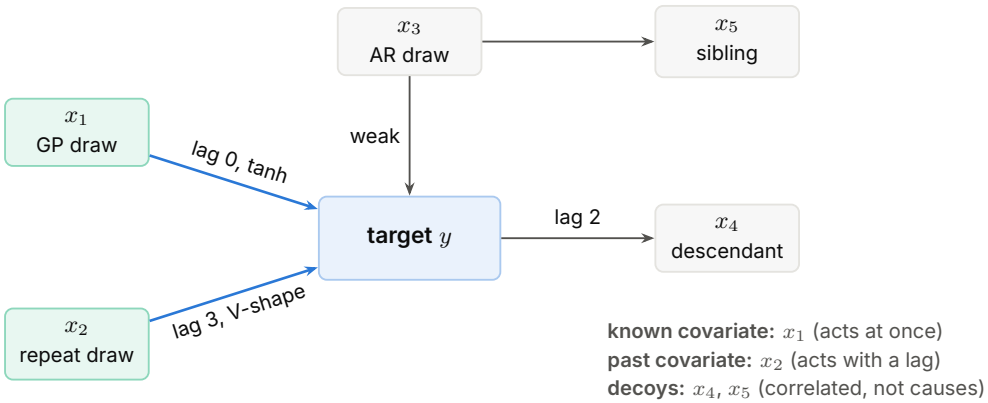


Figure 10: One draw. The target's parents become covariates (known if the effect is immediate, past if lagged); a descendant and a sibling go in as decoys that correlate with the target but do not help forecast it.

Each node is a base-family series; each edge has a lag and a transfer function:

$$x_j(t) \;=\; f_j\Big(\textstyle\sum_{i \in \mathrm{pa}(j)} w_{ij}\, g_{ij}\big(x_i(t - \ell_{ij})\big)\Big) + \varepsilon_j(t),$$

with g drawn from linear, saturating, threshold, V-shaped and multiplicative shapes, and f adding the node's own dynamics.

> **Keep in mind** t0 (beta and gamma) uses a generator of this kind, but I have not seen an ablation of its effect. Treat it as its own switch, tested alone.

6.3 A calendar engine

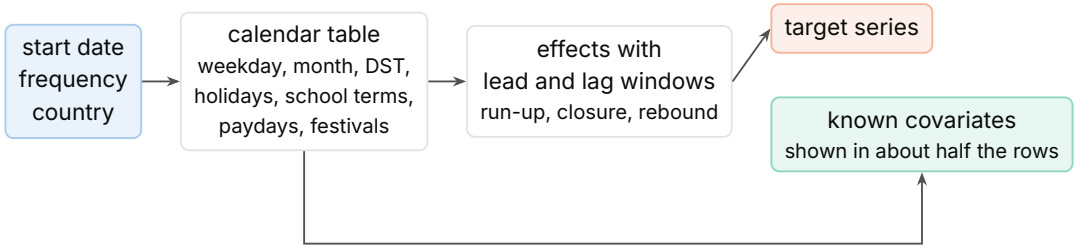


Figure 11: The same real calendar shapes the target and, in half of the rows, is handed to the model as known covariates. The model learns both to read calendar inputs and to cope without them.

7 Real data for checking our work

Tuning the generator against fev or TIME would quietly turn them into training data. So we built a separate catalog of real datasets that are in none of the three benchmarks: **377 datasets across 26 domains**, from power grids and fuel prices to bacterial growth curves, library checkouts and hard-drive failures.

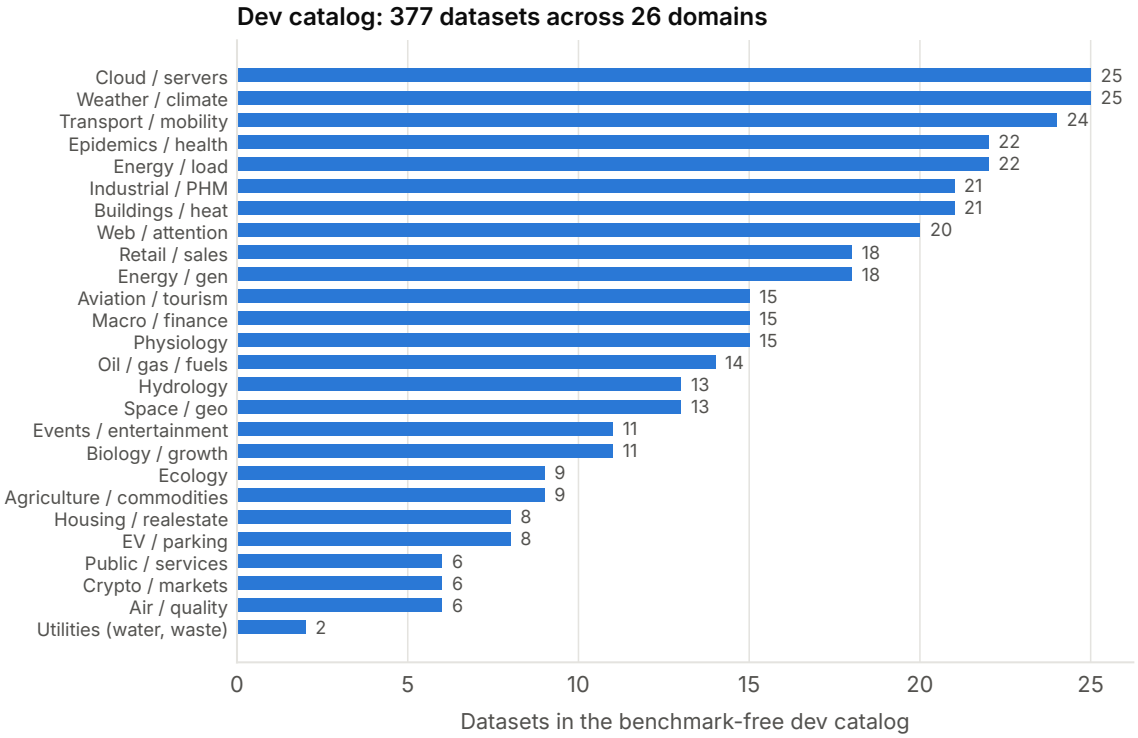


Figure 12: Datasets per domain in `dataset_catalog_v2.csv`. 23 catalog entries were removed because they are benchmark data or the same series under another publisher; 29 are kept with named sub-series excluded.

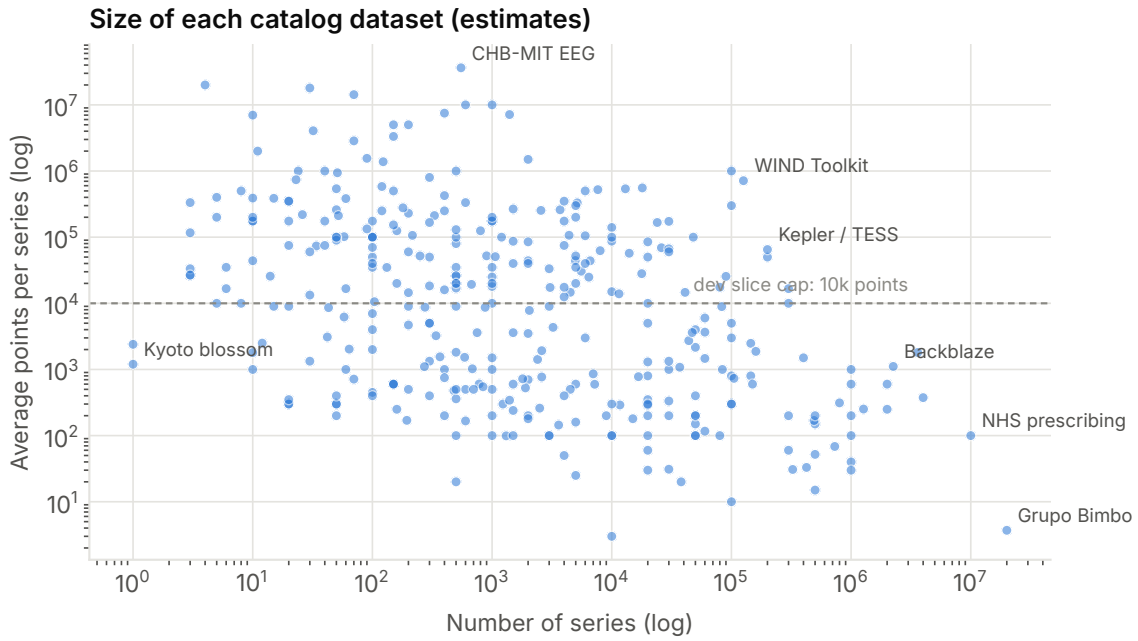


Figure 13: Estimated size of each dataset. The full catalog holds about 62 million series; the planned dev slice takes at most 500 series and 10,000 points from each dataset, about 132,000 series and 650 million points.

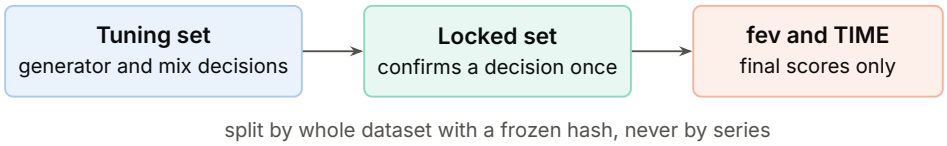


Figure 14: Three roles for data. A dataset sits in exactly one role for the whole project.

About the catalog

- Sizes are order-of-magnitude estimates. 148 URLs were confirmed by search; the rest were written from memory, and `verify_urls.py` checks them all on your machine.
- What counts statistically is the number of *datasets*, not series: 100,000 series from one source are still one source. Cap each dataset's share.
- After downloading, run a series-level check (hashes of values) against the benchmarks, not just a name check.

8 Finding out why a forecast fails

A rising loss tells us *that* something went wrong, not *why*. We find the reason in four steps.

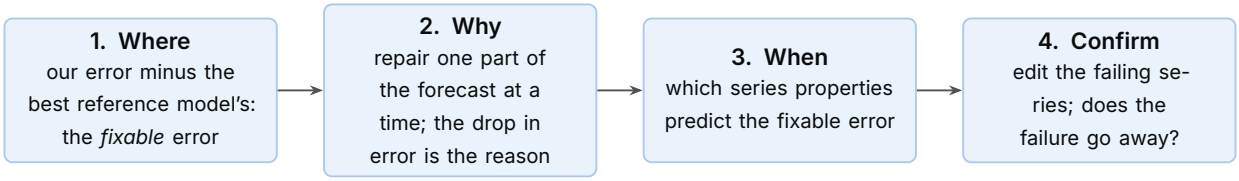


Figure 15: The diagnosis loop. Only a confirmed cause turns into a generator or model change.

8.1 Step 1: fixable error

For each window we compare our error with the best of several strong models (Chronos-2, TimesFM, Toto, Timestral) and simple baselines. If every model fails, the window is probably unpredictable and we ignore it.

$$\Delta_{\text{fix}} = e_{\text{ours}} - \min_{m \in \text{references}} e_m.$$

TIME publishes window-level predictions for every model and fev publishes per-task results, so the references cost nothing.

8.2 Step 2: attribution by repair

Take a bad window and replace one part of our forecast with the truth's: its level, its trend, its seasonal amplitude, its phase, its spikes, its interval width. The share of the error that each repair removes is the reason, as a number.

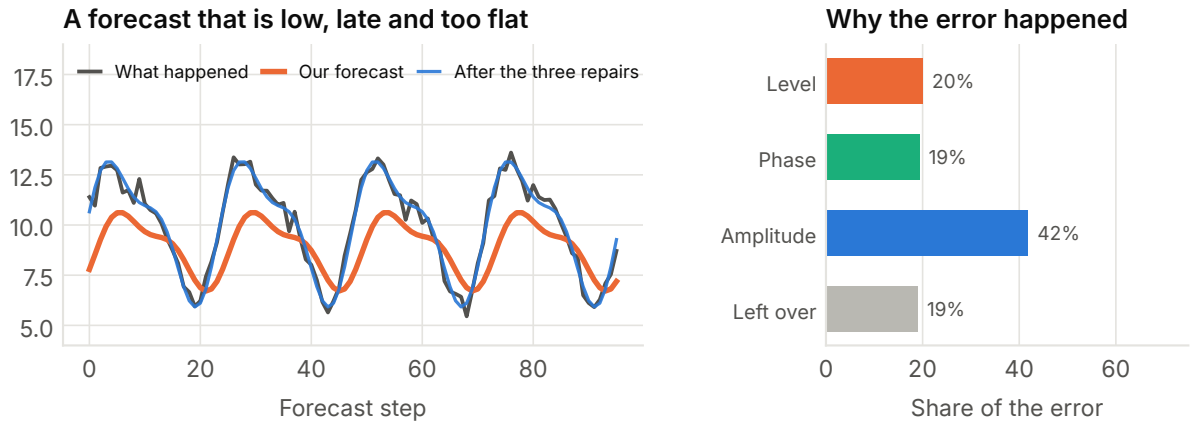


Figure 16: A worked example on a constructed forecast. The forecast is low, two steps late and too flat. Fixing the level removes 20% of the error, the phase 19%, the amplitude 42%; 19% is noise that no repair can remove.

> **Two failures that look the same** "It didn't follow the cycle" can mean the shape is right but late (a **phase** problem), or the timing is right but the swing is too small (an **amplitude** problem: the model is hedging). They have different fixes, and the repair split tells them apart.

8.3 Step 3: which series properties predict failure

Compute features per window: period divided by context length, seasonal and trend strength, intermittency, a level shift near the end of the context, whether covariates exist, frequency, length, noise. Then model the fixable error on these features. The output is a short list of rules such as "we lose when the period is longer than a quarter of the context" or "we lose after a late level shift".

8.4 Step 4: confirm by intervention

Edit a failing real series and forecast again: give it more context, remove the shift, rescale it, add or remove a covariate. If the failure disappears, the cause is found. Then build a controlled synthetic probe for it and keep it as a regression test.

9 Changes on the model side

Better data has a ceiling if the model cannot use it. These sit next to the generator work.

Change	Why	Cost
Known covariates see the whole window	today they see one patch ahead; Timestral lets them see the full forecast window and blocks them from reading the target	fine-tune (masks)
Static and categorical inputs	store type, region, product category have no way in	small
Keep missing positions at inference	the univariate path deletes them and shifts the phase	one line
Longer context (32K) and DHF decoding	the 2049+ history band is our weakest	new pre-training
21 quantiles, advancing refine frame	better tails and long horizons	fine-tune

10 The plan

10.1 First experiment: separate data from model

Timestral-5M's edge is concentrated on covariate tasks (44.5 vs Toto-22m's 37.6, while the two are close without covariates). That could come from data, from covariate attention, or both. Four 5M arms on the same budget answer it:

Arm	What changes	New covariate data	Whole-window covariates
A	current generator	–	–
B	covariate scenarios + causal graph	✓	–
C	known covariates see the whole window	–	✓
D	both	✓	✓

Score them on the dev set's covariate datasets and on the covariate probes, then confirm the winner once at 30M (our DHF test showed that 5M results do not always transfer).

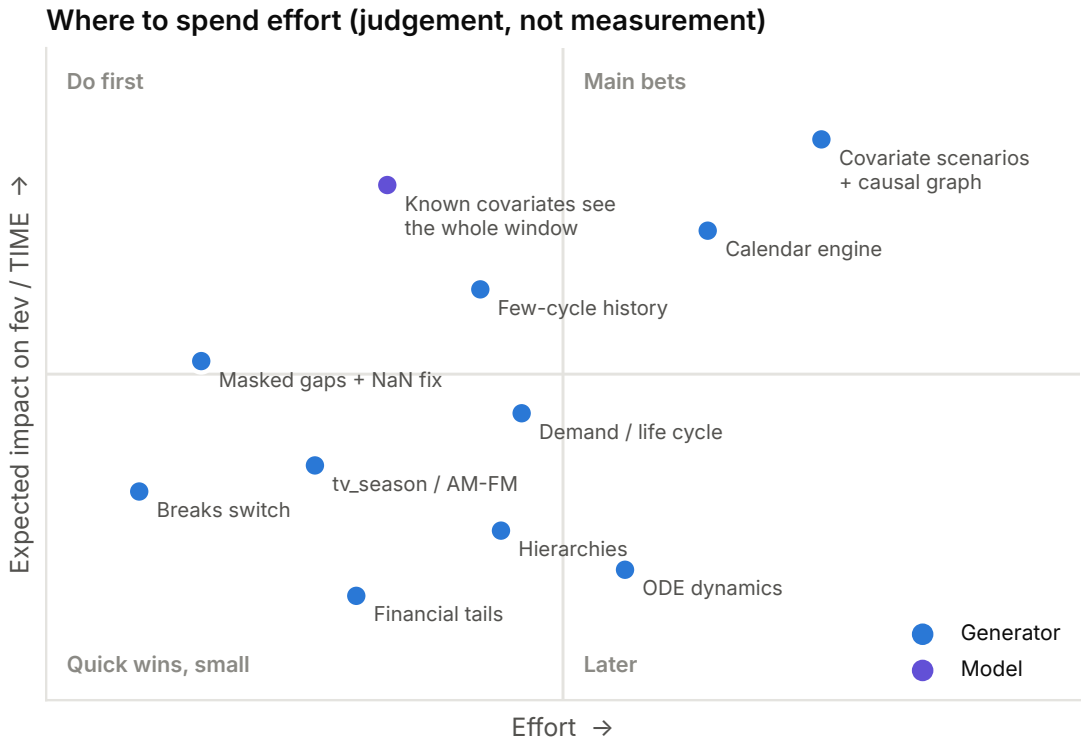


Figure 17: Where effort should go. Positions are judgement, informed by the measurements in this document.

10.2 Order of work

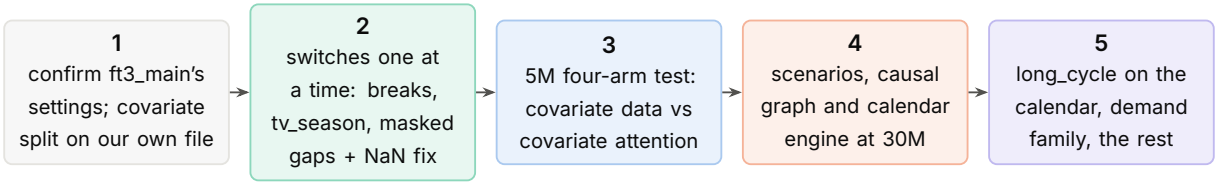


Figure 18: Each step has a probe that isolates it, a dev-set score, and fev and TIME only at the end.

11 Rules to keep

Discipline

1. **Write the check before the run.** Hypothesis, the probe that isolates it, the dev datasets it should move, and the size of change that counts as a win.
2. **One change per fine-tune**, judged with paired confidence intervals.
3. **Keep the three data roles apart** (tuning, locked, final). Count every look at fev and TIME.
4. **Make sure our scorer reproduces 43.78.** The repository's scorer does not have the groups mode yet; until it does, a "we beat X" claim rests on a number the repository cannot regenerate.
5. **Record failures as carefully as successes.** The patch-probe and post-training notes are a good model.
6. **Decide the synthetic-only question now.** The top models post-train on real data. Choose whether

"synthetic only" is part of the claim before the next results arrive, not after.

Usefulness, not just skill Skill is a geometric mean and hides the failures users notice. Report next to it: the share of windows more than twice as bad as Seasonal Naive, interval coverage at 80% and 90%, behaviour with missing values and outliers, stability when one new point arrives, whether covariates help or hurt, and speed per series.

12 Ideas worth exploring

These are not yet in the plan. Each could become a probe or a generator family if the diagnosis points there.

More mathematical generators

- **State-space and pole-zero sampling:** draw the roots of an ARMA or linear system directly to control cycles, damping and memory.
- **Spectral-mixture Gaussian processes:** a few spectral peaks with widths give quasi-periodic structure in one draw.
- **Point processes:** Hawkes, Cox and renewal processes for arrivals, bursts and call volumes.
- **Stochastic differential equations with jumps:** Heston-style volatility, Merton jumps, fractional Brownian motion for long memory.
- **Threshold and smooth-transition AR (TAR, STAR):** regimes that switch on the level itself.
- **Multifractal and wavelet cascades:** bursty, self-similar traffic and turbulence.
- **Dynamic factor models:** many series driven by a few shared factors, the natural shape of retail and macro panels.
- **Observation operators:** censoring, aggregation, rounding, reporting delays and revisions applied after the process is drawn.

Better ways to choose the mix

- **Mixture search on the dev set (DoReMi-style):** learn family weights that reduce dev error, never benchmark error.
- **Coverage maps:** compute features for real and synthetic windows; regions of real data with no synthetic neighbours name the missing family.
- **A realism classifier:** a small model that tells real from synthetic windows; what it finds easy to spot is what the generator gets wrong.
- **Quality-diversity search:** keep generator settings that are both realistic and different from each other.
- **Oracle regret:** for families with a known best forecast (Kalman filter, GP posterior), measure how far the model is from the best possible answer.

Questions for the paper

- Does patch tokenisation gate in-context recall, and do the sub-patch dips follow the periods the training data over-represents? (A causal test: train with periods that avoid multiples of 4.)

- How far can a synthetic-only model go when the covariate mechanisms are generated, not learned from real data?

A Glossary and sources

Skill score	$100\,(1 -$ geometric mean of $e_{\text{model}}/e_{\text{Seasonal Naive}})$ across tasks
MASE, CRPS	point error scaled by a naive forecast; error of the full predicted distribution
Covariate	another series that helps explain the target; known (future given) or past (history only)
Fixable error	our error minus the best reference model's error on the same window
Family	one kind of synthetic process in the generator (for example ets, repeat, long_cycle)
Layer	a change applied on top of any family (seasonality, trend, calendar, noise, corruption)
Pass-through family	a family that skips the layers so its defining property survives (exact repetition, flatness)
Dev catalog	real datasets not in any benchmark, used for tuning and checking
CPM	contiguous patch masking: hiding a run of patches so the model learns to forecast several patches

Sources. fev-bench results: `github.com/autogluon/fev`, `benchmarks/fev_bench/results` and `tasks.yaml`. TIME: `github.com/zqiao11/TIME`, `src/timebench/config/datasets.yaml`; paper arXiv 2602.12147 (dataset sources, Table 2). GIFT-Eval: `github.com/SalesforceAIResearch/gift-eval`. Project notes: TIME_RESULTS, TIME_FULL_LEADERBOARD, TIMESTRAL_GAP, PATCH_PROBE, M4_RESULTS, REAL_POSTTRAIN_RESULTS, DEV_DATA_CATALOG, GENERATOR_GAPS. Generator: `tidemark/real/ft3_core.py`.

Files. `make_figs.py` (all figures), `gen_audit.py` (generator measurements), `dataset_catalog_v2.csv`, `excluded_benchmark.csv`, `benchmark_datasets.csv`, `verify_urls.py`.